Day 51 – JMS Queue Adapter (Java Messaging Service)

Class date: 8th September 2026 | Module: Adapters (JMS), Retry & Decoupling

---

Confirmed against SAP documentation – nothing to correct, plus one nice validation: your note that the Dead Letter Queue mainly matters for **"out of memory" issues** is exactly right and matches SAP's own official guidance almost word for word (Section 8 has the detail). Good instinct to note that down.

1. What is a JMS Queue? (start here first, very important topic)

Messaging Queue Servers – examples: **IBM MQ, Kafka,** etc.

Messaging Queue concept: somebody **publishes** a message to the queue, and somebody else **utilizes/consumes** that message from the queue.

```
System A -> Queue (Messages, a Container) -> System B
```

Key principle: there is **no direct interaction** between System A and System B for the message exchange process – both only ever talk to the Queue, never to each other directly. This is an **Event-driven architecture**, same underlying idea as JMS.

In SAP CPI, we don't use third-party queues (like a standalone IBM MQ or Kafka cluster) for this – instead, we use **JMS Queues internally**, built into **SAP BTP CPI / Integration Suite** itself.

2. Why JMS Queues matter: Retry Mechanism & Data Persistence

JMS Queues give you two things that matter enormously in real projects: **Retry Mechanism** and **Data Persistence**. These are the main reasons JMS is used so heavily in production integrations.

Scenario: iFlow1 places a message onto the Queue (**Push**); iFlow2 picks that message up from the Queue (**Pull**). This is the classic **Push/Pull** pattern between a producer iFlow and a consumer iFlow via a shared Queue.

Decoupling of iFlows via messaging queues: instead of iFlow1 calling iFlow2 directly (**tightly coupled**), both interact only through the Queue (**decoupled**). Retry mechanism and decoupling are the two big reasons we use JMS Queues.

Retry mechanism explained: suppose an interface/iFlow fails – the message will be **reprocessed again** automatically, because it's sitting safely in the JMS Queue rather than being lost.

If you want to use a different/third-party messaging queue (outside SAP CPI's own internal JMS), from SAP CPI to that third-party messaging queue, you use the **AMQP Adapter** instead.

3. Why JMS specifically, in SAP CPI

The main reason JMS is used in SAP CPI: if a message fails inside an iFlow, it will be automatically **retried**. This directly answers the common interview question: **"What do you use for the Retry Mechanism in SAP CPI?"** → **JMS Queues.**

Guarantee & message transfer: JMS Queues follow a **Pub/Sub (Publish/Subscribe) model.**

Demo takeaway: CPI doesn't support a retry mechanism automatically on its own – so we specifically need to use **JMS Queues** to get that capability.

Scenario diagram – iFlow1 pushes the message to a JMS Queue:

```
Sender (HTTPS) -> Integration Process -> JMS Queue iFlow2 then consumes that message from the queue -> Integration Process -> Mail (Receiver)
```

4. Demo – why decoupling matters (the wrong mail server test)

JMS Queue → **Manage Stores** → **Message Queues:** iFlow1 **writes** to the queue; iFlow2 **consumes** from the queue.

Test performed: HTTPS → Mail, with a deliberately wrong Mail server configured – this call **fails** outright (since it's a direct, tightly-coupled call).

But with a JMS Queue in between? iFlow1 (Welcome iFlow1) succeeds and writes the message to the Queue; iFlow2 (Welcome iFlow2) is the one that then tries to send the actual mail and fails (still wrong mail server) – but because the message already safely reached the Queue, **iFlow1 shows as Successful**, and only **iFlow2 fails**. The message isn't lost – it just sits retrying in the Queue until the mail server issue is fixed.

```
Postman -> CPI iFlow -> Queue -> CPI iFlow2 -> Mail
```

Message Queue is always Asynchronous – one-way communication, and the sender does **not wait for a response. Always remember: Queues = Asynchronous.**

This is exactly the value decoupling brings: a downstream failure (bad mail server) doesn't take down the upstream producer (iFlow1) – iFlow1's job was just to reliably get the message into the Queue.

5. The main interview question on this topic

"How will you do retry in CPI?" – this covers error handling, reprocessing failed messages, and guaranteeing delivery from Source to Target.

Answer: JMS Queues – unless/until the message is **successful**, the message **remains in that Queue** (available for retry), rather than being lost after a single failed attempt.

6. JMS Adapter – Receiver, Processing tab

Field	Value / Notes
Queue Name	Any name of your choosing.
Access Type	Exclusive or Non-Exclusive.
Retention Threshold for Alerting (in d)	2 (in the demo; this is also SAP's own default) – if a message stays in the queue longer than this many days, an alert is generated (visible under the Monitor tab).
Expiration Period (in d)	30 in the demo (SAP's own platform default is 90 days) – after this many days, an undelivered message is automatically deleted from the queue.
Compress Stored Message	Enabled in the demo.
Encrypt Stored Message	Enabled in the demo – encrypts the message before storing it.

Transfer Exchange Properties	Enabled – properties stay available inside CPI (properties are scoped to the iFlow only, unlike headers).

Example given for Access Type: if a message queue is being consumed by **10 worker nodes**, **Non-Exclusive** allows **parallel processing** across those nodes; **Exclusive** forces **sequential processing** (one message at a time, one by one).

7. JMS Adapter – Sender, Connection tab

Field	Value / Notes
Queue Name	Any name.
Access Type	Non-Exclusive or Exclusive.
Number of Concurrent Processes	1 in the demo (must be 1 for Exclusive access – exclusive means no parallel consumers are allowed).

Retry Details

Field	Value / Notes
Retry Interval (in min)	1
Exponential Backoff	Enabled – doubles the retry interval after every failed attempt.
Maximum Retry Interval (in min)	60 – this is the ceiling the doubling stops at.
Dead-Letter Queue	Enabled.

Exponential Backoff in action (exactly the sequence in your notes, confirmed correct against SAP's own documentation): starting interval 5 min → doubles each retry → 5, 10, 20, 40, 80 min... capped at the Maximum Retry Interval (60 min) once it would exceed that ceiling. This prevents CPI from hammering a receiver system that's down for a long stretch.

Retry only works with Access Type = Non-Exclusive – because retry relies on parallel processing capability across the queue's consumers, which Exclusive access does not allow.

8. Dead Letter Queue – the real reason it exists

General flow for any message queue: a message hits its **maximum retry** count without succeeding → it moves to the **Dead Letter Queue**, for later manual review.

Your note nailed the deeper, SAP-specific reason for this – confirmed word-for-word against SAP's own official guidance: the Dead Letter Queue exists specifically to prevent **out-of-memory** situations. If a message keeps failing and retrying indefinitely (e.g. because it's unusually large, or the worker node hits a processing problem), it can pile up and cause memory pressure. Moving it to the Dead Letter Queue after repeated retries takes it out of the active retry cycle and protects the tenant's resources. SAP's own recommendation is: only turn the Dead-Letter Queue **off** if you're certain your scenario has small messages with no realistic out-of-memory risk.

Real-time guidance: Unlimited retries are not recommended – always cap retries with a Maximum Retry Interval and a working Dead Letter Queue, rather than letting a message retry forever.

9. Putting it together – an error-routing iFlow pattern

Example iFlow2 structure demonstrated in class, combining JMS with a Router for success/failure handling:

```
Sender -> JMS -> Router Route 1 (success path): Content Modifier -> Mail (Receiver)
-> End Route 2 (failure path): Send -> JMS (a separate Error Queue)
```

This pattern lets successful messages flow on to the real target (Mail), while messages that hit a problem get explicitly routed into a dedicated **Error Queue** for separate handling/monitoring, instead of mixing error messages back into the main processing queue.

Quick-recall summary

- **JMS Queue** = SAP CPI's internal messaging queue (Pub/Sub), used instead of a third-party queue; for actual third-party queues, use the **AMQP Adapter**.
- Two main reasons to use it: **Retry Mechanism** and **Decoupling** of producer/consumer iFlows.
- **CPI has no automatic retry on its own** – JMS Queues are the standard answer to "how do you implement retry in CPI?"
- Queues are always **Asynchronous**, one-way, no waiting for a response.
- **Access Type:** Non-Exclusive = parallel processing across worker nodes; Exclusive = sequential, one-by-one (and forces Concurrent Processes = 1).
- **Exponential Backoff** doubles the retry interval each time (e.g. 5→10→20→40→80 min), capped at the **Maximum Retry Interval.**
- **Dead Letter Queue** exists mainly to prevent **out-of-memory** issues from endlessly-retrying messages – not just a generic "max retries reached" bucket.